Projection in RealSense SDK 2.0

Jump to bottom

Nir Azkiel edited this page on Sep 16 · 7 revisions

This document describes the projection mathematics relating the images provided by the RealSense depth devices to their associated 3D coordinate systems, as well as the relationships between those coordinate systems. These facilities are mathematically equivalent to those provided by previous APIs and SDKs, but may use slightly different phrasing of coefficients and formulas.

Table of Contents

- Pixel Coordinates
- Point Coordinates
- Intrinsic Camera Parameters
  - Distortion Models
- Extrinsic Camera Parameters
- Depth Image Formats
- Processing Blocks Helpers
  - Point Cloud
  - Frame Alignment
- Appendix: Model Specific Details
  - D400 and D500 Series

Pixel Coordinates

Each stream of images provided by this SDK is associated with a separate 2D coordinate space, specified in pixels, with the coordinate `[0,0]` referring to the center of the top left pixel in the image, and `[w-1,h-1]` referring to the center of the bottom right pixel in an image containing exactly `w` columns and `h` rows. That is, from the perspective of the camera, the x-axis points to the right and the y-axis points down. Coordinates within this space are referred to as "pixel coordinates", and are used to index into images to find the content of particular pixels.

Point Coordinates

Each stream of images provided by this SDK is also associated with a separate 3D coordinate space, specified in meters, with the coordinate `[0,0,0]` referring to the center of the physical imager. Within this space, the positive x-axis points to the right, the positive y-axis points down, and the positive z-axis points forward. Coordinates within this space are referred to as "points", and are used to describe locations within 3D space that might be visible within a particular image.

Intrinsic Camera Parameters

The relationship between a stream's 2D and 3D coordinate systems is described by its intrinsic camera parameters, contained in the `rs2_intrinsics` struct. Each model of RealSense device is somewhat different, and the `rs2_intrinsics` struct must be capable of describing the images produced by all of them. The basic set of assumptions is described below:

1. Images may be of arbitrary size

- The `width` and `height` fields describe the number of rows and columns in the image, respectively

2. The field of view of an image may vary

- The `fx` and `fy` fields describe the focal length of the image, as a multiple of pixel width and height

3. The pixels of an image are not necessarily square

- The `fx` and `fy` fields are allowed to be different (though they are commonly close)

4. The center of projection is not necessarily the center of the image

- The `ppx` and `ppy` fields describe the pixel coordinates of the principal point (center of projection)

5. The image may contain distortion

- The `model` field describes which of several supported distortion models was used to calibrate the image, and the `coeffs` field provides an array of up to five coefficients describing the distortion model

Knowing the intrinsic camera parameters of an images allows you to carry out two fundamental mapping operations.

1. Projection

- Projection takes a point from a stream's 3D coordinate space, and maps it to a 2D pixel location on that stream's images. It is provided by the header-only function `rs2_project_point_to_pixel(...)` .

2. Deprojection

- Deprojection takes a 2D pixel location on a stream's images, as well as a depth, specified in meters, and maps it to a 3D point location within the stream's associated 3D coordinate space. It is provided by the header-only function `rs2_deproject_pixel_to_point(...)` .

Intrinsic parameters can be retrieved from any `rs2::video_stream_profile` object via a call to `get_intrinsics()` . (See example here)

Distortion Models

Based on the design of each model of RealSense device, the different streams may be exposed via different distortion models. The full list is the `rs2_distortion` enum in rs_types.h.

1. **None** (`RS2_DISTORTION_NONE`)

- An image has no distortion, as though produced by an idealized pinhole camera. This is typically the result of some hardware or software algorithm undistorting an image produced by a physical imager, but may simply indicate that the image was derived from some other image or images which were already undistorted. Images with no distortion have closed-form formulas for both projection and deprojection, and can be used with both `rs2_project_point_to_pixel(...)` and `rs2_deproject_pixel_to_point(...)` .

2. **Brown-Conrady** Distortion (`RS2_DISTORTION_BROWN_CONRADY`)

- The unmodified Brown-Conrady model (radial `k1..k3` and tangential `p1, p2` coefficients). Projection is closed-form; deprojection is solved iteratively inside `rs2_deproject_pixel_to_point(...)` . The rectified depth and infrared streams of the D400 and D500 series report this model with all coefficients set to zero, i.e. they are effectively undistorted. The D500 series color stream also uses this model.

3. **Modified Brown-Conrady** Distortion (`RS2_DISTORTION_MODIFIED_BROWN_CONRADY`)

- A variation of Brown-Conrady where tangential distortion is applied to the radially distorted points. This model provides a closed-form formula to map from undistorted points to distorted points only, so it is supported by `rs2_project_point_to_pixel(...)` but **not** by `rs2_deproject_pixel_to_point(...)` .

4. **Inverse Brown-Conrady** Distortion (`RS2_DISTORTION_INVERSE_BROWN_CONRADY`)

- The coefficients describe the inverse mapping (distorted to undistorted). Both directions are supported: deprojection is closed-form, projection is solved iteratively. This model is used by the color stream of the D400 series.

5. **F-Theta** (`RS2_DISTORTION_FTHETA`)

- Fish-eye model where the image radius is proportional to the incidence angle. Supported by both projection and deprojection. Used by wide-FOV fisheye imagers.

6. **Kannala-Brandt** (`RS2_DISTORTION_KANNALA_BRANDT4`)

- Four-parameter Kannala-Brandt fish-eye model. Supported by both projection and deprojection.

RealSense devices always support deprojection from depth images, and always support projection to color images. Therefore, it is always possible to map a depth image into a set of 3D points (a point cloud), and it is always possible to discover where a 3D object would appear on the color image.

Extrinsic Camera Parameters

The 3D coordinate systems of each stream may in general be distinct. For instance, it is common for depth to be generated from one or more infrared imagers, while the color stream is provided by a separate color imager. The relationship between the separate 3D coordinate systems of separate streams is described by their extrinsic parameters, contained in the `rs2_extrinsics` struct. The basic set of assumptions is described below:

1. Imagers may be in separate locations, but are rigidly mounted on the same physical device

- The `translation` field contains the 3D translation between the imager's physical positions, specified in meters

2. Imagers may be oriented differently, but are rigidly mounted on the same physical device

- The `rotation` field contains a 3x3 orthonormal rotation matrix between the imager's physical orientations

3. All 3D coordinate systems are specified in meters

- There is no need for any sort of scaling in the transformation between two coordinate systems

4. All coordinate systems are right handed and have an orthogonal basis

- There is no need for any sort of mirroring/skewing in the transformation between two coordinate systems

Knowing the extrinsic parameters between two streams allows you to transform points from one coordinate space to another, which can be done by calling `rs2_transform_point_to_point(...)`. This operation is defined as a standard affine transformation using a 3x3 rotation matrix and a 3-component translation vector.

Extrinsic parameters can be retrieved via a call to `rs2_get_extrinsics(...)` between any two streams which are supported by the device, or using a `rs2::stream_profile` object via `get_extrinsics_to(...)` (see example here). One does not need to enable any streams beforehand, the device extrinsics are assumed to be independent of the content of the streams' images and constant for a given device for the lifetime of the program.

Depth Image Formats

As mentioned above, mapping from 2D pixel coordinates to 3D point coordinates via the `rs2_intrinsics` structure and the `rs2_deproject_pixel_to_point(...)` function requires knowledge of the depth of that pixel in meters. Certain pixel formats, exposed by this SDK, contain per-pixel depth information, and can be immediately used with this function. Other images do not contain per-pixel depth information, and thus would typically be projected into instead of deprojected from.

1. `RS2_FORMAT_Z16` (Under `rs2_format`)

- Depth is stored as one unsigned 16-bit integer per pixel, mapped linearly to depth in camera-specific units. The distance, in meters, corresponding to one integer increment in depth values can be queried via `rs2_get_depth_scale(...)` or using a `rs2::depth_sensor` via `get_depth_scale()` (see example here).

  The following shows how to retrieve the depth of a pixel in meters:

  Using `C` API:

```
const float scale = rs2_get_depth_scale(sensor, NULL);
const uint16_t * image = (const uint16_t *)rs2_get_frame_data(frame, NULL);
float depth_in_meters = scale * image[pixel_index];
```

  Using `C++` API:

```
rs2::depth_frame dpt_frame = frame.as<rs2::depth_frame>();
float pixel_distance_in_meters = dpt_frame.get_distance(x,y);
```

  using `Python` API:

```
dpt_frame = pipe.wait_for_frames().get_depth_frame().as_depth_frame()
pixel_distance_in_meters = dpt_frame.get_distance(x,y)
```

- If a device fails to determine the depth of a given image pixel, a value of zero will be stored in the depth image. This is a reasonable sentinel for "no depth" because all pixels with a depth of zero would correspond to the same physical location, the location of the imager itself.
- The default scale of a D400 or D500 series device is one millimeter, allowing for a maximum expressive range of ~65 meters. The depth scale can be modified by calling `rs2_set_option(...)` with `RS2_OPTION_DEPTH_UNITS`, which specifies the number of meters per one increment of depth. 0.001 would indicate millimeter scale, while 0.01 would indicate centimeter scale.

Processing Blocks Helpers

The SDK provides two main processing blocks related to image projecting:

1. Point Cloud
2. Frame Alignment

Point Cloud

As part of the API we offer a processing block for creating a point cloud and corresponding texture mapping from depth and color frames. The point cloud created from a depth image is a set of points in the 3D coordinate system of the depth stream. The following demonstrates how to create a point cloud object:

Using `C` API:

```
rs2_frame_queue* q = rs2_create_frame_queue(1, NULL);
rs2_processing_block* pc = rs2_create_pointcloud(NULL);
rs2_start_processing_queue(pc, q, NULL);
rs2_process_frame(pc, depth_frame, NULL);
rs2_frame* points = rs2_wait_for_frame(q, 5000, NULL);
const rs2_stream_profile* sp = rs2_get_frame_stream_profile(color_frame, NULL);
rs2_get_stream_profile_data(sp, ..., &uid, ..., NULL);
rs2_set_option((rs2_options*)pc, RS2_OPTION_TEXTURE_SOURCE, uid, NULL);
rs2_process_frame(pc, color_frame, NULL);
```

Using `C++` API:

```
rs2::pointcloud pc;
rs2::points points = pc.calculate(depth_frame);
```

```
pc.map_to(color_frame);
```

using `Python` API:

```
import pyrealsense2 as rs
pc = rs.pointcloud()
points = pc.calculate(depth_frame)
pc.map_to(color_frame)
```

For additional examples, see examples/pointcloud.

In addition, an example for integrating with PCL (Point Cloud Library) is available here.

Frame Alignment

Usually when dealing with color and depth images, mapping each pixel from one image to the other is desired. The SDK offers a processing block for aligning the image to one another, producing a set of frames that share the same resolution and allow for easy mapping of pixels.

The following demonstrates how to create an `align` object:

Using `c` API:

```
rs2_processing_block* align = rs2_create_align(RS2_STREAM_COLOR, NULL);
rs2_frame_queue* q = rs2_create_frame_queue(1, NULL);
rs2_start_processing_queue(align, q, NULL);
rs2_process_frame(align, depth_and_color_frameset, NULL);
rs2_frame* aligned_frames = rs2_wait_for_frame(q, 5000, NULL);
for (i = 0; i < rs2_embedded_frames_count(aligned_frames, NULL); i++)
{
    rs2_frame* aligned_frame = rs2_extract_frame(aligned_frames, i, NULL);
}
```

Using `C++` API:

```
rs2::align align(RS2_STREAM_COLOR);
auto aligned_frames = align.process(depth_and_color_frameset);
rs2::video_frame color_frame = aligned_frames.first(RS2_STREAM_COLOR);
rs2::depth_frame aligned_depth_frame = aligned_frames.get_depth_frame();
```

using `Python` API:

```
import pyrealsense2 as rs
align = rs.align(rs.stream.color)
aligned_frames = align.process(depth_and_color_frameset)
```

```
color_frame = aligned_frames.first(rs.stream.color)
aligned_depth_frame = aligned_frames.get_depth_frame()
```

For additional examples, see examples/align and align-depth2color.py.

Appendix: Model Specific Details

It is not necessary to know which model of RealSense device is plugged in to successfully make use of the projection capabilities of this SDK. However, developers can take advantage of certain known properties of given devices.

D400 and D500 Series

1. Left and right infrared images are rectified by default (Y16 format is not)

- The two infrared streams have identical intrinsics
- The two infrared streams have no distortion
- There is no rotation between left and right infrared images (identity matrix)
- There is translation on only one axis between left and right infrared images (`translation[1]` and `translation[2]` are zero)
- Therefore, the `y` component of pixel coordinates can be used interchangeably between these two streams

2. Stereo disparity is related to depth via an inverse linear relationship, and the distance of a point which registers a disparity of 1 can be queried via `1 / rs2_get_depth_scale(...)` . The following shows how to retrieve the depth of a pixel in meters:

```
const float scale = rs2_get_depth_scale(sensor, NULL);
const uint16_t * image = (const uint16_t *)rs2_get_frame_data(frame, NULL);
float depth_in_meters = 1 / (scale * image[pixel_index]);
```

- Unlike `RS2_FORMAT_Z16` , a disparity value of zero is meaningful. A stereo match with zero disparity will occur for objects "at infinity", objects which are so far away that the parallax between the two imagers is negligible. By contrast, there is a maximum possible disparity.
- Disparity output is available on the D400 series and is controlled via the `advanced_mode` API.

▸ Pages 10

RealSense SDK Wiki

- Home
- Troubleshooting Q&A
- Release Notes
- API Changes
- API How-To
- Build Configuration
- Realsense Viewer Manual
- Overview of Depth from Stereo
- Post-Processing
- Sample Data for SDK examples
- D400 Series Visual Presets
- D400 Advanced Mode
- Projection in RealSense SDK 2.0
- API Documentation
- Record and Playback
- About Android Support
- RealSense with Raspberry Pi 3
- RealSense with Raspberry Pi 4
- RealSense with NVidia Jetson
- ROS 2 Wrapper
- Frame Buffering Management
- Supported Matrix
- Docker

Help

- Closed GitHub Issues
- Support
- Open a new GitHub issue

Clone this wiki locally

```
https://github.com/realsenseai/librealsense.wiki.git
```